ARQUITECTURA DE MENSAJERÍA • GUÍA TÉCNICA

RabbitMQ Cluster con Docker Compose & Spring Boot

Operación, Validación de Erlang Cookie, Clustering Manual y Monitoreo Desatendido



1. El Secreto Fundamental: La Cookie de Erlang

RabbitMQ se ejecuta sobre el runtime distribuido de Erlang. Para que los nodos formen un clúster unificado, deben autenticarse mediante un secreto compartido denominado **Erlang Cookie**. Si los valores difieren, los nodos rechazarán cualquier conexión inter-cluster por motivos de seguridad.

Propósito Clave

Actúa como firma criptográfica en el puerto 25672. Garantiza que solo miembros autorizados puedan sincronizar metadatos del broker.

Ubicación Interna

El daemon de arranque copia la variable de entorno a `/var/lib/rabbitmq/.erlang.cookie` con permisos estrictos de lectura (0400).

2. Verificación de la Cookie en Contenedores

Compruebe que ambos contenedores comparten exactamente el mismo átomo de Erlang en tiempo de ejecución:

```
# Consulta directa al kernel de Erlang en ambos nodos: docker exec rabbitmq-1 rabbitmqctl eval 'erlang:get_cookie().' docker exec rabbitmq-2 rabbitmqctl eval 'erlang:get_cookie().' # Salida esperada en ambas terminales: 'secret_cookie_value'
```

Aviso sobre el archivo de configuración (.conf vs .config)

Asegúrese de que el archivo en disco se llame estrictamente `rabbitmq.conf` y no `rabbitmq.config`. RabbitMQ 3.8+ usa la sintaxis sysctl. Asimismo, mantenga comentada la línea `management.load_definitions` salvo que monte un archivo JSON existente.

3. Procedimiento de Unión Manual al Clúster

Si por condiciones de carrera el nodo 2 inicializó una base de datos Mnesia independiente antes del nodo 1, ejecute esta secuencia para federar el clúster en caliente sin destruir contenedores:

```
# 1. Detener la app RabbitMQ en nodo 2 (mantiene el contenedor vivo) docker exec rabbitmq-2 rabbitmqctl stop_app # 2. Resetear el estado local de Mnesia (limpia nodos independientes) docker exec rabbitmq-2 rabbitmqctl reset # 3. Unir formalmente nodo 2 al cluster liderado por nodo 1 docker exec rabbitmq-2 rabbitmqctl join_cluster rabbit@rabbitmq-1 # 4. Reanudar el motor de mensajería docker exec rabbitmq-2 rabbitmqctl start_app
```

4. Monitoreo Desatendido y Diagnóstico Continuo

Para validar el flujo continuo sin interrumpir su flujo de desarrollo, mantenga terminales dedicadas con seguimiento en tiempo real:

Logs en Vivo (Follow Mode)

Supervisa conexiones AMQP, declaraciones y handshakes en tiempo real:

```
docker-compose logs -f rabbitmq-1
docker-compose logs -f rabbitmq-2
```

Estado del Clúster

Verifica que ambos nodos figuren en `Running Nodes` y `Disk Nodes`:

```
docker exec rabbitmq-1 \
rabbitmqctl cluster_status
```

5. Integración con Spring Boot & Pruebas de Flujo

La aplicación gestiona recursos mediante `RabbitAdmin` y administra listeners dinámicamente con `RabbitListenerEndpointRegistry`:

Acción	Comando HTTP (cURL)	Comportamiento Observado
Crear Cola	`POST /api/rabbitmq/queues?queueName=orders.queue`	Spring declara la cola; aparece sincronizada en UI :15672 y :15673.
Listar Listeners	`GET /api/listeners`	Retorna `["order-listener"]` registrado por Spring Boot.
Pausar Consumo	`POST /api/listeners/order-listener/pause`	Invoca `container.stop()`; consumidores en UI bajan a 0.
Reanudar Consumo	`POST /api/listeners/order-listener/resume`	Invoca `container.start()`; procesa mensajes pendientes acumulados.

Validación Visual en Consola Web

Acceda a **http://localhost:15672** y **http://localhost:15673** (credenciales `admin/admin123`). En la pestaña *Overview* > *Nodes* ambos nodos deben figurar con métricas verdes sin alertas de red partition.